SEUPD@CLEF: Team NEXA

Paul Arlot[1], Andrea Di Tillo[1], Gaute Greiff Flagstad[1], Bita Khashechian[1], Danil Smirnov[1] and Marco Tomaiuoli[1]

[1]*University of Padua, Italy*

Abstract

This paper presents the system developed by Team NEXA for Task 1 of the CLEF 2026 CheckThat! Lab, focusing on the retrieval of scientific sources for claims found on social media. Given the challenge of linking informal web discourse to formal scientific literature across English, German, and French, we propose a multi-stage retrieval framework. Our approach combines traditional lexical search using BM25 with a neural re-ranking component based on multilingual transformer models. The system aims to bridge the linguistic and stylistic gap between social media posts and academic abstracts, providing a scalable solution to assist fact-checkers in verifying scientific information.

Keywords

Information Retrieval, Search Engine, CLEF, CheckThat!, Scientific Claim Verification, Lucene, Multilingual Search

1. Introduction

Information Retrieval systems play a fundamental role in enabling access to relevant data across massive, often heterogeneous corpora. In the context of the CheckThat! Lab at CLEF, which focuses on identifying the original scientific research behind social media claims, we have developed a configurable IR system. Our system supports a wide range of preprocessing and indexing strategies, enabling us to experiment with different setups to address the vocabulary gap between informal web text and formal scientific abstracts.

The motivation for this project comes from the need to fight disinformation by making fact-checking faster. As noted by CheckThat!, manually searching for scientific evidence is very slow for human experts. By automating this source retrieval process, we can help fact-checkers quickly link a claim to its original paper. To achieve this, we compare two methods consisting of a **language-based approach** using specific tools for English, German, and French, and a **general approach** that treats all languages the same. Our goal is to find the best way to accurately match multilingual claims to their scientific sources.

The paper is organized as follows: Section 2 describes our approach; Section 3 explains our experimental setup; Section 4 discusses our main findings; finally, Section 5 draws some conclusions and outlooks for future work.

2. Methodology

In this section, we describe the architecture and components of our system. Our approach follows a multi-stage Information Retrieval pipeline combining traditional lexical matching with semantic representations.

"Search Engines", course at the master degree in "Computer Engineering", Department of Information Engineering, and at the master degree in "Data Science", Department of Mathematics "Tullio Levi-Civita", University of Padua, Italy. Academic Year 2025/2026

paullouisjean.arlot@studenti.unipd.it (P. Arlot); andrea.ditillo@studenti.unipd.it (A. D. Tillo); gaute.greiff.flaegstad@studenti.unipd.it (G. G. Flagstad); bita.khashechian@studenti.unipd.it (B. Khashechian); danil.smirnov@studenti.unipd.it (D. Smirnov); marco.tomaiuoli@studenti.unipd.it (M. Tomaiuoli)

2.1. System Configurability

2.2. Document Parsing

2.3. Query Parsing

2.4. Query Expansion

2.5. Text Analysis

The text analysis phase transforms raw textual input into a stream of tokens that can be indexed and later retrieved. The analysis is performed through language-specific analyzer classes, which extend the Apache Lucene `Analyzer` framework. In our system, we provide separate analyzers for each supported language, namely `EnglishAnalyzer`, `FrenchAnalyzer`, and `GermanAnalyzer`. Each analyzer is configured using language-dependent resources and processing pipelines.

All analyzers follow a common architecture composed of three main components:

- **Tokenizer:** Responsible for splitting the input text into raw tokens.
- **Token Filters:** Sequentially modify, normalize, or enrich the token stream.
- **Stemming/Lemmatization:** Reduces tokens to their base or root forms to improve generalization.

The overall pipeline is dynamically configured through external YAML configuration files, allowing flexible adaptation of processing strategies for different languages and experimental settings.

2.5.1. Tokenizer Configurations

Multiple tokenization strategies are supported, enabling the system to handle different types of input text:

- **Standard Tokenizer:** Uses Lucene's rule-based tokenizer to handle punctuation, digits, and word boundaries.
- **Letter Tokenizer:** Emits tokens composed only of alphabetic characters, treating non-letter characters as delimiters.
- **Whitespace Tokenizer:** Splits input strictly on whitespace characters without processing punctuation.
- **NLP Tokenizer:** Uses Apache OpenNLP models to perform linguistically informed tokenization.

The tokenizer type is selected through configuration files, allowing different languages or datasets to use the most suitable strategy.

2.5.2. Token Filters

Token filters are applied after tokenization to normalize and enrich the token stream. These filters are categorized into general (language-independent) and language-specific filters.

General Token Filters These filters perform general normalization operations across all languages:

- **LowerCaseFilter:** Converts all tokens to lowercase.
- **ICUFoldingFilter:** Normalizes accented and special characters into ASCII equivalents.
- **NBSPFilter:** Removes non-breaking spaces and trims tokens.
- **RemoveDuplicatesTokenFilter:** Eliminates duplicate tokens.
- **LengthFilter:** Removes tokens that are too short or too long.
- **RepeatedLetterFilter:** Normalizes tokens with repeated characters (e.g., "soooo" $\rightarrow$ "soo").

Enrichment Filters To improve semantic representation, additional filters are applied:

- **AbbreviationExpansionFilter:** Expands abbreviations using predefined mappings (e.g., "ml" → "machine learning").
- **CompoundPOSTokenFilter:** Uses Part-of-Speech tagging to combine semantically related tokens into compound expressions.
- **ShingleFilter:** Generates n-grams to capture local contextual information.
- **PositionFilter:** Normalizes positional increments for indexing consistency.

Language-Specific Filters Each language analyzer applies additional filters tailored to its linguistic characteristics:

- English-specific filters (e.g., possessive removal, English stopwords)
- French-specific filters (e.g., elision handling, French stopwords)
- German-specific filters (e.g., compound handling and normalization)

This design ensures that each language is processed according to its grammatical and lexical properties.

2.5.3. Stemming and Lemmatization

The system supports multiple strategies for reducing tokens to their canonical forms: **Porter Stemmer**, **Snowball Stemmer**, **OpenNLP Lemmatizer**, **No Stemming**.

The selected method is controlled through configuration files and can vary depending on the language and experimental setup.

2.5.4. Multi-Language Analyzer Design

A key feature of the system is the use of separate analyzers for each language. Each analyzer loads its own resources, including: Language-specific stopword lists, Synonym dictionaries, Abbreviation mappings, OpenNLP models (tokenizer, POS tagger, lemmatizer).

These resources are organized in a structured directory hierarchy (e.g., `resources/en`, `resources/fr`, `resources/de`), ensuring clear separation between languages.

This design allows the system to handle multilingual corpora effectively, as each language benefits from tailored preprocessing and linguistic enrichment.

Overall, the integration of language-specific analyzers, configurable processing pipelines, and rich linguistic resources allows the system to effectively bridge the gap between informal social media text and formal scientific documents, improving the quality of retrieval.

2.6. Document Indexing

The document indexing stage transforms processed publications into a searchable inverted index using Apache Lucene. This structure enables efficient retrieval of relevant documents during the query phase. In our system, indexing is handled by the `DirectoryIndexer` component, which coordinates parsing, preprocessing, and storage of documents.

2.6.1. Indexing Workflow

The indexing workflow operates over a collection of JSON files containing scientific publications. Each file is parsed to extract individual documents, represented internally as structured `Publication` objects. Only documents containing meaningful textual content (such as title or abstract) are considered for indexing.

A key aspect of the system is its multilingual processing capability. For each publication, the language is automatically detected, and an appropriate language-specific analyzer is applied. Separate analyzers are used for English, French, and German, each configured with its own tokenization and filtering pipeline.

In addition to standard preprocessing, the system supports optional enhancements. When enabled, documents written in languages different from the target language can be translated prior to indexing. Moreover, the system can generate dense vector representations of the textual content by interacting with an external embedding service. These embeddings are stored alongside the indexed documents to support future semantic retrieval extensions.

Each processed publication is converted into a Lucene `Document` and added to the index using an `IndexWriter`. During this process, statistics such as the number of indexed documents and total indexing time are recorded.

2.6.2. Field Representation

Document content is stored in structured fields, such as title and abstract, each processed using the appropriate analyzer based on the detected language. A custom field type is defined to support advanced indexing features, including term frequencies, positional information, and character offsets.

The system also enables storage of term vectors, which allows more advanced retrieval functionalities such as phrase matching, proximity queries, and potential re-ranking strategies. Additionally, the original textual content can be stored in the index to support downstream processing steps.

2.6.3. Processing Pipeline

The indexing procedure follows a well-defined sequence of steps. First, the system initializes the index writer with the configured analyzer and output directory. Then, for each document, the following operations are performed:

- Parsing the input file into structured publication objects
- Filtering out incomplete or invalid entries
- Detecting the language of the document
- Applying optional translation if required
- Generating semantic embeddings (if enabled)
- Converting the document into a Lucene-compatible format
- Adding the document to the index

Once all documents have been processed, the index is finalized by committing changes and closing the writer.

This modular and extensible design allows the indexing system to integrate multiple preprocessing strategies, multilingual handling, and semantic enrichment, while keeping the core indexing logic independent and reusable.

2.7. Retrieval Phase

The retrieval phase is responsible for matching input claims with relevant scientific publications stored in the indexed collection. This component is implemented through the `Searcher` class, which performs query processing, executes search operations over the Lucene index, and outputs ranked results.

The system initializes a Lucene `IndexSearcher` over the indexed data and employs the BM25 similarity function for ranking. Queries are provided as a collection of claims stored in JSON format, where each claim represents a short textual statement that must be linked to relevant documents.

2.7.1. Query Formulation

Each claim is processed independently. The textual content is first transformed into a structured query using a `QueryBuilder`, which applies the same analyzer used during indexing. This ensures that both documents and queries undergo consistent preprocessing, reducing mismatches due to tokenization or normalization.

The resulting terms are used to construct field-specific queries targeting different parts of the indexed documents.

2.7.2. Structured Boolean Queries

Instead of relying on a single-field representation, the system generates separate subqueries for the title and abstract fields of each document.

These subqueries are combined using a Boolean query with SHOULD clauses, allowing documents to be retrieved if they match either field. This formulation increases recall while maintaining flexibility in matching.

To further refine the ranking, different importance weights are assigned to each field through boosting. In particular, matches occurring in the title field are given higher importance than those in the abstract. This results in a weighted query structure that prioritizes concise and informative matches while still considering broader contextual information.

2.7.3. Ranking Mechanism

Once the Boolean query is constructed, it is executed using the Lucene search engine. The system retrieves the top-k documents according to their BM25 scores. This ranking function considers the frequency of query terms, their rarity across the collection, and document length normalization.

The number of retrieved documents is configurable and can be adjusted depending on the evaluation setup.

2.7.4. Result Formatting

The retrieved results are written to an output file following the TREC evaluation format. Each entry includes the query identifier, document identifier, rank position, score, and run identifier. This standardized format ensures compatibility with external evaluation tools commonly used in information retrieval benchmarks.

3. Experimental Setup

Describe the experimental setup, i.e.

- used collections
- evaluation measures
- url to git repository and its organization
- hardware used for experiments
- ...

4. Results and Discussion

Provide a summary of the performance on the previous year dataset.

Discuss the results and any relevant issues.

5. Conclusions and Future Work

Provide a summary of what are the main achievements and findings.

Discuss future work, e.g. what you may try next and/or how your approach could be further developed.

6. Group Members Contribution

Jane Doe Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

John Doe Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Jane Doe Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

John Doe Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Jane Doe Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

John Doe Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

7. Misc [TO BE REMOVED]

7.1. Tex Files

Put your LaTeXfiles into the `section` folder as shown in the examples above.

7.2. Figures

Put your figures into the `figure` folder and put the caption under the figure. Example of reference to Figure 1.

Figure 1: 1907 Franklin Model D roadster. Photograph by Harris & Ewing, Inc. [Public domain], via Wikimedia Commons. (https://goo.gl/VLCRBB).

Table 1
Frequency of Special Characters

Non-English or Math	Frequency	Comments
Ø	1 in 1,000	For Swedish names
π	1 in 5	Common in math
\$	4 in 5	Used in business
Ψ_1^2	1 in 40,000	Unexplained usage

7.3. Tables

Put the caption above the table. Example of reference to Table 1.

See the `booktab` packaged documentation for further options.

7.4. Bibliography

Example of citations:

- name: Salton [1]
- parenthesis: [1]

An initial list of references is provided in the files `bibliography.bib` and `proceedings.bib` that you can expand.

See the `natbib` packaged documentation for further options.

7.5. Acronyms

Use the

`\ac{acronym}`

command to insert acronyms, eg. *Average Precision (AP)*. The command will expand the acronym the first time it is used.

An initial list of acronyms is provided in the file `acronyms.tex` that you can expand.

See the `acronym` packaged documentation for further options.

References

[1] G. Salton, Automatic Information Organization and Retrieval., McGraw-Hill, New York, USA, 1968.